Practical – Week6

Session 6:

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
int main() {
    char message[100];
    int fd_read, fd_write;
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);
    printf("Server started...\n");
    while (1) {
        fd_read = open(FIFO1, O_RDONLY);
        read(fd_read, message, sizeof(message));
        close(fd_read);
        printf("Client: %s\n", message);
        // Exit request
        if (strcmp(message, "exit") == 0) {
            break;
        }
        // Process message
        char response[150];
       snprintf(response, sizeof(response),
                "Server processed: %s", message);
        // Open FIFO for writing
        fd_write = open(FIFO2, O_WRONLY);
        write(fd_write, response, strlen(response) + 1);
        close(fd_write);
   }
    unlink(FIFO1);
    unlink(FIFO2);
    printf("Server stopped.\n");
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
int main() {
    char message[100];
    char response[150];
    int fd_write, fd_read;
    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0';
    // Open FIFO for writing
    fd_write = open(FIFO1, O_WRONLY);
    write(fd_write, message, strlen(message) + 1);
    close(fd_write);
    // Exit request
    if (strcmp(message, "exit") == 0) {
        return 0;
   }
    // Open FIFO for reading
    fd_read = open(FIFO2, O_RDONLY);
    read(fd_read, response, sizeof(response));
    close(fd_read);
    printf("Server response: %s\n", response);
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"
int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s message\n", argv[0]);
        return 1;
    }

    char response[150];

    int fd_write, fd_read;

    // Send message to server
    fd_write = open(FIFO1, O_WRONLY);

    write(fd_write, argv[1], strlen(argv[1]) + 1);

    close(fd_write);

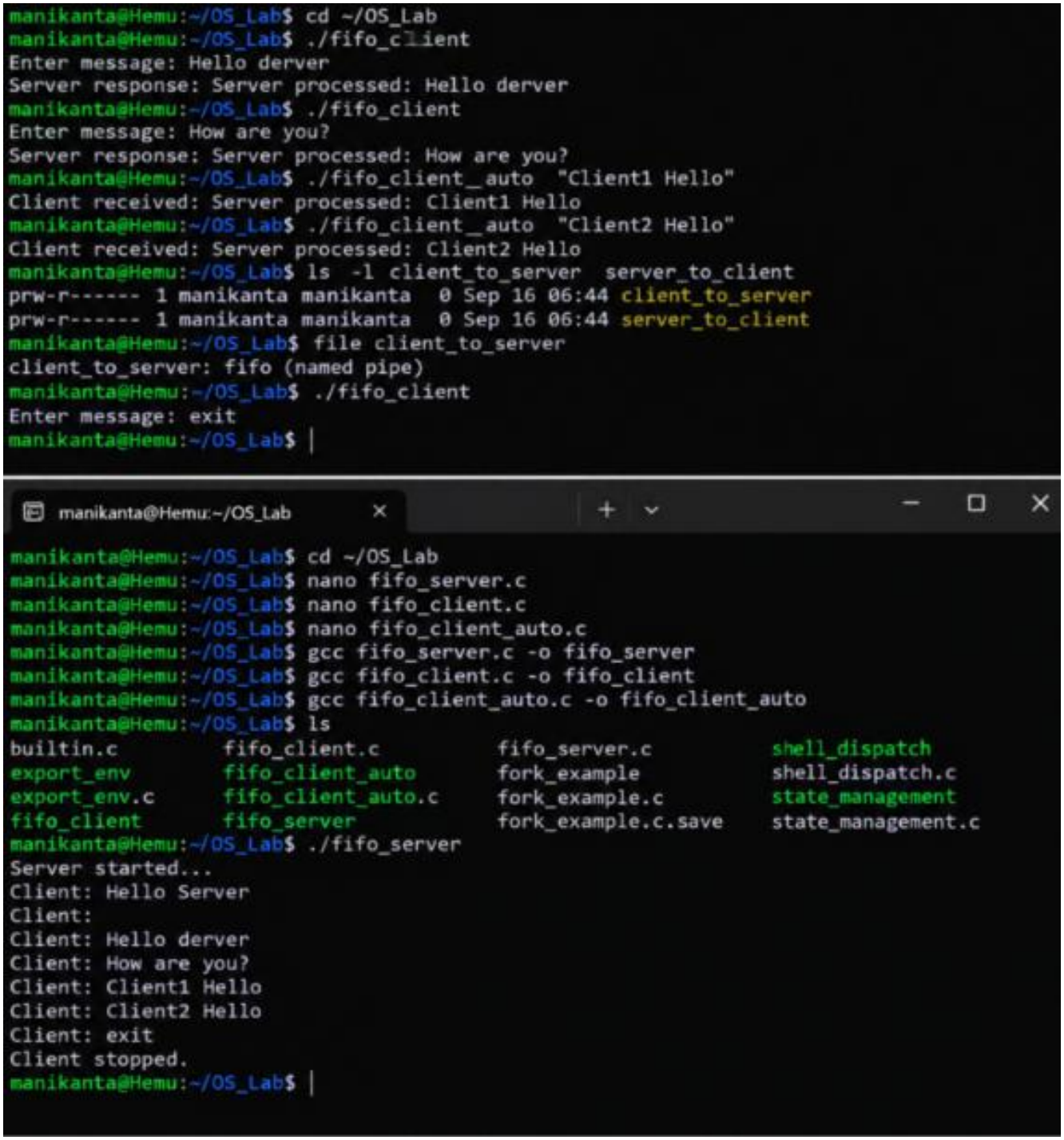
    // Receive response
    fd_read = open(FIFO2, O_RDONLY);

    read(fd_read, response, sizeof(response));

    close(fd_read);

    printf("Client received: %s\n", response);

    return 0;
}
```



Report: The program successfully implements client-server communication using named pipes (FIFOs). The client sends messages, and the server processes and responds to them. Multiple clients can communicate through the FIFO, demonstrating inter-process communication and FIFO behavior.

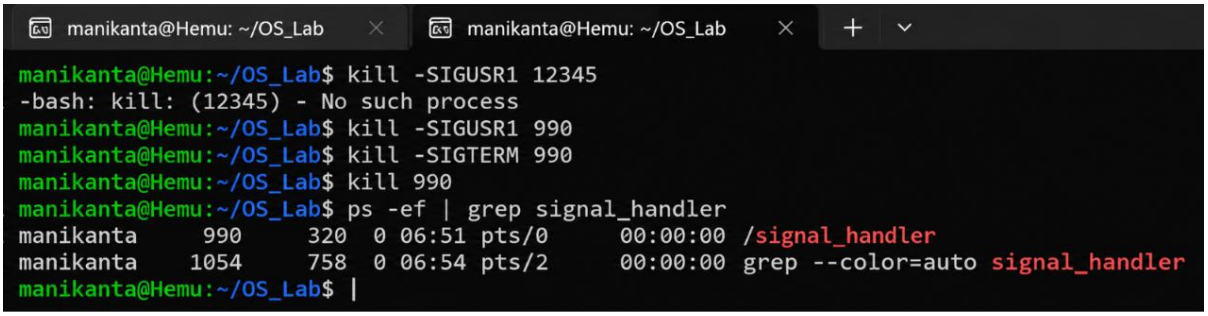
2. Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

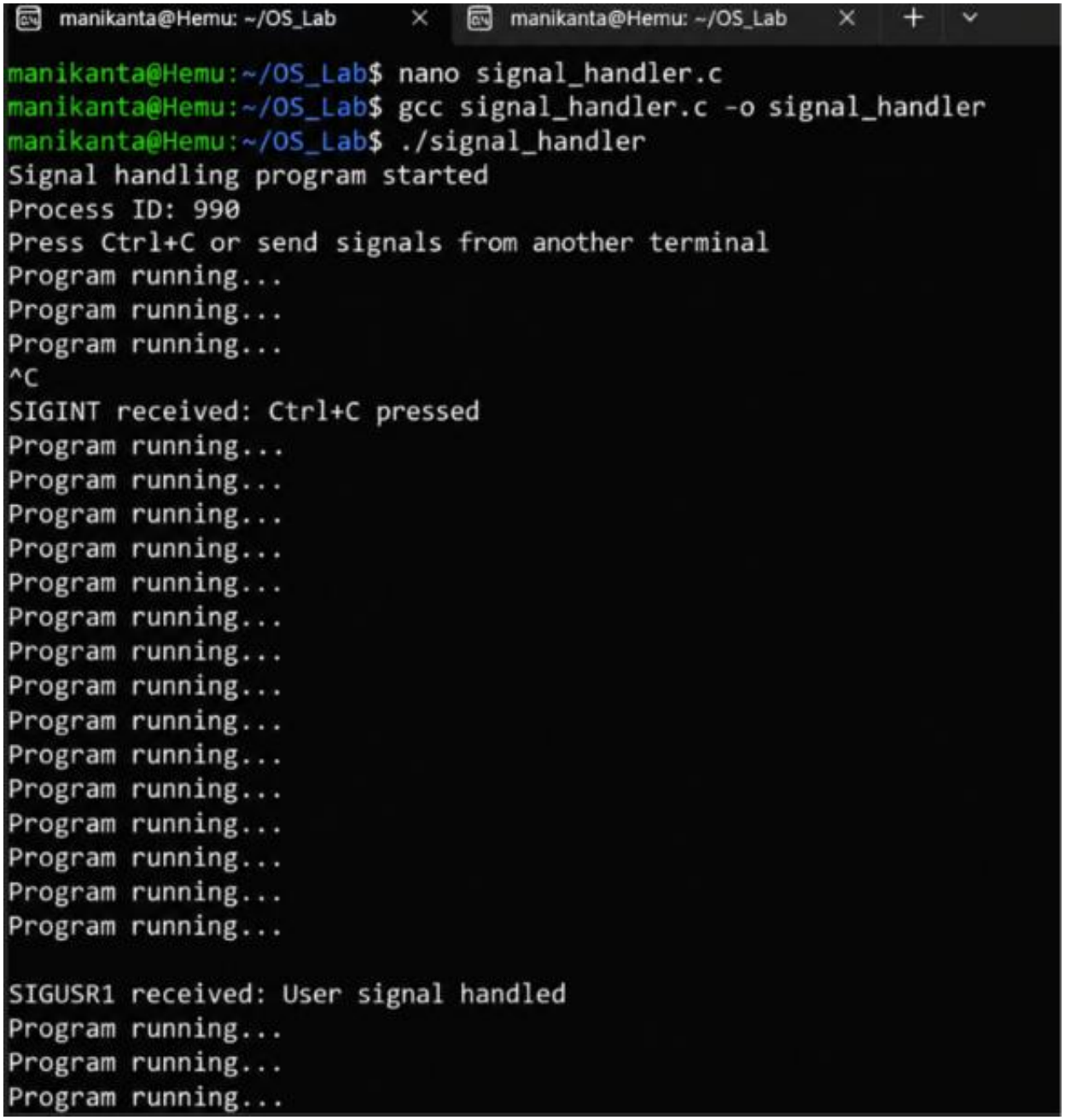
```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
// Signal handler function
void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("\nSIGINT received: Ctrl+C pressed\n");
    }
    else if (sig == SIGTERM) {
        printf("\nSIGTERM received: Termination request\n");
    }
    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received: User signal handled\n");
    }
    fflush(stdout);
}
int main() {
    // Register signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);
    printf("Signal handling program started\n");
    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C or send signals from another terminal\n");

    while (1) {

        printf("Program running...\n");
        sleep(3);
    }

    return 0;
}
```




manikanta@Hemu: ~/OS_Lab
manikanta@Hemu: ~/OS_Lab
manikanta@Hemu:~/OS_Lab$ nano signal_handler.c
manikanta@Hemu:~/OS_Lab$ gcc signal_handler.c -o signal_handler
manikanta@Hemu:~/OS_Lab$ ./signal_handler
Signal handling program started
Process ID: 990
Press Ctrl+C or send signals from another terminal
Program running...
Program running...
Program running...
^C
SIGINT received: Ctrl+C pressed
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
SIGUSR1 received: User signal handled
Program running...
Program running...
Program running...


SIGTERM received: Termination request
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
Program running...
^Z
[1]+ Stopped ./signal_handler

Report: The program successfully captures SIGINT, SIGTERM, and SIGUSR1 using signal handlers. It demonstrates asynchronous event handling by responding to signals while running, making signal handling simple and reliable.